AEGIS: 임베디드 소스코드 정적·동적 분석 플랫폼

초안 버전: 소개용 + 논문화 전략 포함

---

1. 한 줄 소개

AEGIS는 임베디드 소프트웨어를 대상으로, 정적 분석 결과·코드 구조·보안 지식 그래프·실시간 취약점 사례 검색을 결합해 더 신뢰 가능한 취약점 triage와 exploitability 분석을 수행하는 분석 플랫폼이다.

조금 더 직접적으로 말하면, AEGIS는 다음과 같은 문제를 해결하려는 프로젝트다.

- SAST 도구만으로는 결과가 너무 많고 노이즈가 크다.
- LLM만으로 코드를 보게 하면 그럴듯한 분석은 가능하지만, 근거 없는 주장과 환각이 섞이기 쉽다.
- "SAST 결과를 그냥 LLM에 넣는" 단순 하이브리드 방식은 코드 구조, 실제 호출 맥락, 취약점 사례 맥락, exploitability 판단을 충분히 반영하지 못한다.
- 임베디드 소프트웨어는 빌드 문맥, 크로스 컴파일 환경, 하드웨어 의존성, 제한된 런타임 관찰 환경 때문에 일반 웹/앱 분석보다 더 어렵다.

AEGIS는 이 문제를 해결하기 위해 **결정론적 분석을 먼저 수행하고, 그 결과를 근거 중심으로 조직한 뒤, 에이전트가 제한된 역할 안에서 reasoning과 exploitability 해석을 수행하게 만드는 구조**를 지향한다.

---

2. 왜 필요한가

오늘날 소스코드 보안 분석은 대체로 세 방향으로 흘러간다.

1. **전통적 SAST 중심 접근**
    - 정밀한 룰 기반 또는 데이터플로우 기반 분석이 가능하다.
    - 그러나 false positive가 많고, 실제 exploitability를 잘 설명하지 못한다.
    - 여러 도구를 동시에 돌리면 coverage는 늘지만, 결과 통합과 해석 비용이 급증한다.
2. **LLM-only 또는 LLM-heavy 접근**
    - 자연어 설명과 코드 reasoning은 강하다.
    - 그러나 근거가 약하거나, 실제 코드 구조와 빌드 문맥을 놓칠 수 있다.
    - 임베디드 환경처럼 현실적인 제약이 강한 상황에서는 과도하게 heuristic해지기 쉽다.
3. **SAST + LLM 요약형 하이브리드**
    - 일정 수준의 실용성은 있다.
    - 하지만 여전히 "경고 묶음을 요약하고 분류하는 수준"에 머무르는 경우가 많다.
    - 왜 위험한지, 어떤 사례와 연결되는지, 실제 공격 가능성이 있는지, 어떤 PoC가 성립 가능한지를 체계적으로 설명하기 어렵다.

AEGIS는 이 중 어느 한쪽을 극단적으로 택하지 않는다.
핵심 철학은 다음과 같다.

> **기계적으로 확정할 수 있는 것은 최대한 앞단에서 결정론적으로 처리하고, 그 위에서만 에이전트가 근거 기반 추론을 수행하게 만든다.**

즉, AEGIS의 차별점은 "LLM을 더 똑똑하게 쓰는 것"이 아니라, **LLM이 함부로 말할 수 없도록 전체 분석 파이프라인을 설계하는 것**에 있다.

---

3. 이 프로젝트가 기존 방식과 다른 점

3.1 단순 LLM 코드 리뷰가 아니다

AEGIS는 "코드 전체를 LLM에 넣고 취약점을 찾아봐" 식의 접근을 지향하지 않는다.
그 방식은 빠른 데모는 가능하지만, 다음 문제가 있다.

- 분석 근거가 불명확하다.
- 빌드 설정, 전처리 조건, include 구조, 실제 호출 체인, 타 라이브러리와의 연결이 반영되지 않기 쉽다.
- 추후 재현 가능한 평가가 어렵다.

AEGIS는 먼저 **코드, 빌드, 호출 관계, 정적 분석 결과**를 구조화한다.
LLM은 이 구조화된 근거를 바탕으로만 reasoning한다.

3.2 단순 SAST 결과 요약기가 아니다

AEGIS는 "여러 SAST 도구의 결과를 모아서 LLM이 정리해준다" 수준에서 끝나지 않는다.
그보다 더 중요한 것은 **서로 다른 분석 결과를 하나의 보안 주장(claim)으로 승격시키는 기준**이다.

예를 들어, 어떤 함수에 대해 다음 정보가 동시에 잡힐 수 있다.

- 위험 함수 사용
- 사용자 입력에서 해당 함수로 이어지는 호출 체인
- 경계값 검증 부재
- 유사 CWE 패턴
- 과거 유사 CVE 사례
- 해당 약점을 실제 공격 전술이나 CAPEC 패턴으로 연결하는 근거

AEGIS는 이런 정보를 조합해 "경고"를 "근거가 있는 취약점 후보"로 재구성한다.

3.3 일반 소프트웨어가 아니라 임베디드 소프트웨어를 겨냥한다

임베디드 환경은 일반 애플리케이션과 다르다.

- 빌드가 까다롭다.

- 대상 아키텍처가 다르다.
- 실행 환경이 로컬 호스트가 아닐 수 있다.
- 외부 입력은 파일이 아니라 프레임, 패킷, 장치 인터페이스일 수 있다.
- 동적 관찰은 종종 QEMU, 시뮬레이터, 디버거, 트레이서가 필요하다.

AEGIS는 처음부터 이 점을 고려한다.
즉, 단순 코드 진단이 아니라 **임베디드 소프트웨어의 현실적인 보안 분석 워크플로우를 플랫폼 수준에서 정리하는 것**이 목적이다.

---

4. 핵심 아이디어

AEGIS의 핵심 아이디어는 네 줄로 요약할 수 있다.

1. **정적 분석은 많이 하되, 그냥 많이 보여주지 않는다.**
2. **분석 결과는 코드 구조와 보안 지식 그래프로 재조직한다.**
3. **에이전트는 근거를 찾고 연결하는 역할을 하며, 임의 추측을 줄인다.**
4. **최종적으로는 exploitability와 동적 확인까지 이어지는 페루프를 만든다.**

이를 시스템 관점에서 보면 다음과 같은 구조가 된다.

---

5. 시스템 개요

Phase 0. 소스코드 수집 및 빌드 문맥 확보

가장 먼저 필요한 것은 "코드 파일"이 아니라 **분석 가능한 프로젝트 상태**다.

여기서 확보해야 할 것은 다음과 같다.

- 프로젝트 구조
- 빌드 설정
- include 관계
- 사용 라이브러리
- 전처리 매크로
- 크로스 컴파일 여부
- 대상 아키텍처 정보

임베디드 프로젝트는 이 단계가 특히 중요하다.
왜냐하면 빌드 문맥을 모른 채 보는 코드는 실제 동작 코드와 다를 수 있기 때문이다.

Phase 1. 결정론적 정적 분석 오케스트레이션

이 단계에서는 여러 SAST 도구를 최대한 잘 엮어서 돌린다.

목표는 두 가지다.

- 서로 다른 도구가 잡아내는 약점을 가능한 넓게 수집한다.
- 단순 결과 덤프가 아니라 **후속 reasoning에 쓸 수 있는 구조화된 evidence**를 만든다.

이 단계에서 다룰 수 있는 정보 예시는 다음과 같다.

- 위험 API 사용
- 데이터 흐름 추적
- 호출 관계
- 경계 검사 누락
- 버퍼 길이 계산 오류 가능성
- 정수 오버플로우 가능성
- 포인터 안전성 문제
- 라이프사이클 오용
- SCA 수준의 외부 라이브러리 취약점 힌트

핵심은, 이 단계가 **LLM 앞단의 확정 가능한 분석면**이라는 점이다.

Phase 2. 근거 중심 에이전트 분석

이제 에이전트가 등장한다.
하지만 여기서의 역할은 "모든 걸 추론하는 만능 분석가"가 아니다.

에이전트는 다음을 수행한다.

- 수집된 evidence 간 관계 정리
- 서로 다른 경고를 하나의 취약점 후보로 통합
- 관련 CWE/ATT&CK/CAPEC 탐색
- 현재 코드 상황과 가장 유사한 CVE 사례 배치 조회
- 취약점 후보의 exploitability reasoning
- 우선순위 판단
- 분석 보고서 작성

중요한 점은, 에이전트의 모든 최종 주장에는 **코드 근거, 도구 근거, 그래프 근거, 사례 근거**가 연결되어야 한다는 것이다.

즉, AEGIS는 "잘 말하는 AI"를 만드는 것이 아니라,
근거를 추적할 수 있는 보안 분석 주장 생성기를 만드는 쪽에 더 가깝다.

Phase 3. PoC 생성

정적 분석과 사례 검색으로 "위험해 보인다"는 결론이 나왔다고 해서 그것만으로 충분하지는 않다.

AEGIS는 그 다음 단계로 다음을 노린다.

- 입력 제어 가능성 검토
- 취약 지점 도달 경로 정리
- 필요한 조건 명시

- 제약 조건을 반영한 PoC 초안 생성

여기서 PoC는 무조건 공격 성공을 보장하는 것이 아니라,
해당 취약점 후보가 실제 트리거 가능한지 점검하는 테스트 자산으로 보는 것이 맞다.

즉, PoC 생성의 목적은 과장된 공격 시연이 아니라
exploitability를 더 엄밀하게 검토할 수 있는 검증 입력 생성이다.

Phase 4. 동적 확인 (향후 확장)

궁극적으로 AEGIS가 지향하는 것은 정적 의심(suspicion)에서 끝나는 것이 아니라,
정적 후보를 동적으로 확인(confirmation)하는 단계까지 포함한 페루프다.

예상되는 방향은 다음과 같다.

- 정적으로 찾은 취약 지점에 대해
- 생성한 PoC나 테스트 입력을
- QEMU나 유사 시뮬레이션 환경에서 주입하고
- 다른 에이전트나 분석 모듈이 GDB/트레이서/로깅 도구로 런타임 행동을 관찰하고
- 실제로 충돌, 오염, 비정상 분기, 메모리 이상, 보안 속성 위반 등이 발생하는지 확인한다.

이 단계가 붙으면 AEGIS는 단순 분석 플랫폼이 아니라
정적 추론과 동적 검증을 연결하는 embedded exploitability validation framework로 발전한다.

---

6. 이 프로젝트가 지향하는 가치

AEGIS가 노리는 핵심 가치는 세 가지다.

6.1 더 높은 신뢰도

문제는 "많이 찾는 것"이 아니라 "믿을 수 있게 찾는 것"이다.
AEGIS는 근거 없는 취약점 나열보다 **정당화 가능한 취약점 후보**를 더 중요하게 본다.

6.2 더 낮은 진입장벽

임베디드 보안 분석은 원래 어렵다.

- 여러 SAST 도구를 다뤄야 하고
- 결과를 해석해야 하고
- 취약점 taxonomy를 이해해야 하고
- 유사 사례를 찾아야 하며
- 가능하면 exploitability도 봐야 한다

AEGIS는 이 복잡한 파이프라인을 플랫폼으로 묶어
숙련 분석가의 워크플로우를 보다 접근 가능한 형태로 제공하는 것을 목표로 한다.

6.3 더 실제적인 분석

많은 도구가 "이론적으로 위험"에서 끝난다.
AEGIS는 그보다 더 나아가 **실제 코드, 실제 빌드 문맥, 실제 사례, 실제 트리거 가능성**을 함께 다루려 한다.

---

7. 무엇을 주장하지 않을 것인가

이 프로젝트를 소개할 때는, 아래를 과장해서 말하지 않는 편이 좋다.

- "AEGIS는 모든 취약점을 자동으로 찾아준다"
- "AEGIS는 사람 없이 완전 자율적으로 exploit까지 수행한다"
- "LLM이 있으니 기존 정적 분석은 필요 없다"
- "누구나 버튼 한 번으로 완벽한 보안 감사를 할 수 있다"

더 정확한 표현은 다음과 같다.

- AEGIS는 **기존 정적 분석을 대체하는 것이 아니라, 그 결과를 더 강한 형태로 재구성한다.**
- AEGIS는 **분석 자동화를 높이되, 근거 추적성과 검증 가능성을 유지하려 한다.**
- AEGIS는 **false positive를 줄이고 exploitability 판단을 더 실제적으로 만드는 방향**을 노린다.
- AEGIS의 장점은 "만능"이 아니라 **구조화와 근거화**에 있다.

---

8. 왜 이 프로젝트가 논문이 될 수 있는가

프로젝트를 만드는 것과 논문을 쓰는 것은 다르다.
논문은 플랫폼 소개가 아니라 **측정 가능한 주장**이어야 한다.

AEGIS는 플랫폼 전체로 보면 큰 엔지니어링 프로젝트지만,
논문으로는 다음과 같이 **서로 다른 두 편**으로 자르는 것이 가장 자연스럽다.

---

9. 논문 A1: Deterministic-First Evidence-Grounded Vulnerability Triage for Embedded C/C++

9.1 문제 정의

임베디드 C/C++ 프로젝트 분석에서는 다음 두 문제가 동시에 발생한다.

- 전통적 SAST는 결과가 많지만 false positive가 크고 exploitability 판단이 약하다.
- LLM 기반 분석은 자연어 reasoning은 가능하지만 근거 없는 주장과 불안정성이 크다.

즉, 현재의 문제는 "탐지가 되느냐"보다
"실제로 믿고 triage할 수 있는 취약점 후보를 얼마나 안정적으로 만들 수 있느냐"에 있다.

9.2 핵심 주장

이 논문의 중심 주장은 다음과 같다.

> **결정론적 분석을 앞단에 두고, 모든 최종 취약점 주장을 코드/도구/지식 근거에 연결하도록 강제하면, embedded C/C++ 프로젝트에서 naive LLM 또는 단순 SAST+LLM 방식보다 더 높은 precision과 더 낮은 grounding error를 달성할 수 있다.**

9.3 이 논문에서의 핵심 기여

- 임베디드 C/C++ 취약점 triage를 위한 deterministic-first 파이프라인 제안
- 다중 SAST, 코드 구조, 위험 호출 맥락, knowledge graph, live CVE lookup을 결합한 evidence model 제안
- "찾았다"가 아니라 "왜 그렇게 판단했는지 근거를 추적 가능한 claim" 구조 설계
- grounding error, triage precision, analyst burden 감소를 중심으로 한 평가 프레임 제시

9.4 무엇을 범위에 넣고, 무엇을 빼야 하는가

이 논문에서는 **동적 분석은 과감히 빼는 것이 좋다.**

이 논문은 다음에 집중해야 한다.

- 취약점 후보를 더 정확하게 재구성할 수 있는가
- false positive를 줄일 수 있는가
- 근거 없는 분석 결과를 줄일 수 있는가

반대로 아래는 다음 논문으로 넘긴다.

- 실제 페이로드 주입 실험
- 디버거 기반 동적 확인
- 완전한 exploit 성공 여부

즉, A1은 **정적 triage 논문**으로 깔끔하게 가는 편이 강하다.

9.5 평가 설계

데이터셋

- 빌드 가능한 embedded 또는 low-level C/C++ 프로젝트
- 오픈소스 firmware, automotive-adjacent code, protocol stack, device library 등
- known vulnerability가 포함된 프로젝트 + 수동 검증 샘플

비교 대상

- SAST union only
- LLM-only 코드 분석

- SAST 결과를 요약해 LLM에 넣는 단순 하이브리드
- AEGIS full pipeline

핵심 지표

- Precision
- Recall
- F1
- False Discovery Rate
- Grounding Error Rate
- Time-to-First-Valid-Finding
- Analyst review effort
- Cost / latency

Ablation

- Knowledge graph 제거
- Live CVE lookup 제거
- 위험 호출자 추적 제거
- 근거 강제 규칙 제거

9.6 예상 목차

1. Introduction
2. Background and Problem Setting
3. Limitations of SAST-only and LLM-heavy Triage
4. AEGIS A1 Design
5. Evidence-Grounded Claim Construction
6. Experimental Setup
7. Results
8. Ablation Study
9. Limitations and Discussion
10. Related Work
11. Conclusion

9.7 왜 논문이 되는가

이 논문의 공헌은 "플랫폼을 만들었다"가 아니다.
진짜 공헌은 다음 문장으로 표현되어야 한다.

> **근거를 강제하는 deterministic-first orchestration이 embedded vulnerability triage의 정확도와 신뢰도를 실제로 개선한다.**

즉, 이것은 단순 엔지니어링이 아니라
보안 분석 시스템 설계 원리와 평가 프레임을 제시하는 논문이 된다.

9.8 현실적인 포지셔닝

이 논문은 "세상을 완전히 바꾸는 만능 플랫폼"보다
실제 분석가가 쓸 수 있는 더 믿을 만한 triage 구조로 포지셔닝하는 것이 맞다.

---

10. 논문 A2: Static-to-Dynamic Exploitability Confirmation for Cross-Compiled Embedded Software

10.1 문제 정의

정적 분석으로 취약점 후보를 찾았다고 해서 그것이 실제 exploitability를 의미하지는 않는다.
특히 임베디드 환경에서는 다음 문제가 크다.

- 빌드는 되더라도 실행 환경이 다르다.
- 실제 입력 전달 경로가 로컬 프로세스와 다르다.
- 정적으로는 위험해 보여도 런타임에서는 제약 때문에 성립하지 않을 수 있다.
- 반대로 정적 분석이 과소평가한 경로가 런타임에서 실제로 열릴 수 있다.

즉, 핵심 문제는
정적 분석 결과를 실제 exploitability 판단으로 연결하는 마지막 단계가 비어 있다는 것이다.

10.2 핵심 주장

> **정적 후보 위치를 기반으로 QEMU/디버거/관찰 도구를 이용한 동적 확인을 수행하면, embedded cross-compiled software에서 actionable finding의 precision을 크게 높일 수 있다.**

10.3 이 논문의 핵심 기여

- 정적 취약점 후보를 동적 확인 대상으로 승격하는 selection pipeline
- PoC 또는 테스트 입력을 실제 실행 환경에 주입하는 검증 루프
- 런타임 관찰을 통해 exploitability를 재판정하는 dynamic confirmation framework
- "정적으로 위험해 보임"과 "동적으로 확인됨"을 분리한 평가 체계

10.4 이 논문에서 중요한 점

이 논문은 A1과 다르게 **실행 검증**이 핵심이다.
따라서 아래 요소가 꼭 필요하다.

- 크로스 컴파일 대상 실행 방법
- sandboxed environment
- 입력 주입 방법
- 관찰 기준

- 성공/실패 판정 기준

PoC는 이 논문에서 단순 텍스트 산출물이 아니라
동적 검증을 위한 실행 가능한 테스트 자산이어야 한다.

10.5 평가 설계

데이터셋

- 재현 가능한 크로스컴파일 프로젝트
- known vulnerability가 있거나, 수동으로 exploitability가 검증 가능한 샘플
- 가능하면 category별 샘플 확보
  - buffer misuse
  - integer issue
  - parser flaw
  - unsafe state machine transition
  - input validation failure

비교 대상

- Static-only triage
- Static triage + PoC generation without execution feedback
- Full AEGIS static-to-dynamic loop

핵심 지표

- PoC compile success
- PoC execution success
- Dynamic confirmation rate
- Actionable finding precision
- False-confirm rate
- Human intervention count
- End-to-end analysis time

10.6 예상 목차

1. Introduction
2. Why Static Suspicion Is Not Enough
3. System Model and Threat Model
4. Static-to-Dynamic Confirmation Pipeline
5. Runtime Observation and Decision Criteria
6. Experimental Setup
7. Results
8. Failure Analysis
9. Limitations, Safety, and Scope

10. Related Work
11. Conclusion

10.7 왜 논문이 되는가

이 논문이 논문이 되는 이유는 "PoC도 만든다"가 아니다.

핵심은 다음이다.

> **정적 의심을 동적 확인으로 연결하는 구조가 embedded exploitability 판단의 실효성을 높인다.**

즉, 이는 취약점 탐지 논문이 아니라
취약점 검증과 확인의 자동화 수준을 높이는 시스템 논문으로 볼 수 있다.

10.8 안전성과 윤리

이 논문은 반드시 다음 범위를 분명히 해야 한다.

- 분석 대상은 소유 또는 허가된 코드/환경에 한정
- 네트워크 상의 무단 공격이 아니라 로컬 또는 실험용 샌드박스 실행
- exploitability 확인은 검증 목적이며, 배포 악용이 목적이 아님
- 결과 공개 시 재현성과 안전성의 균형 고려

---

11. AEGIS 논문화 순서

가장 현실적인 순서는 다음과 같다.

1단계: 논문 A1

먼저 정적 triage 논문부터 쓴다.

이유:

- 이미 구현이 더 많이 되어 있을 가능성이 높다.
- 평가 경계가 명확하다.
- 논문 claim이 단단하다.
- 동적 인프라 없이도 성과를 보여줄 수 있다.

2단계: 논문 A2

그다음 static-to-dynamic confirmation으로 확장한다.

이유:

- A1의 결과를 그대로 A2의 앞단으로 사용할 수 있다.

- 동적 검증은 구현 비용이 크므로 별도 논문으로 분리하는 편이 좋다.
- 두 논문이 논리적으로도 자연스럽게 이어진다.

즉, AEGIS는 한 편으로 크게 쓰는 것보다
A1(정적 triage) → A2(동적 confirmation)
의 2단 분할이 가장 논문답다.

---

12. 이 프로젝트를 소개할 때의 추천 문장

짧은 소개

AEGIS는 임베디드 소프트웨어 분석에서 다중 SAST, 코드 구조 분석, 보안 지식 그래프, 사례 기반 retrieval을 결합해 더 신뢰 가능한 취약점 triage와 exploitability 분석을 수행하는 플랫폼이다.

조금 긴 소개

AEGIS는 단순히 SAST 결과를 LLM에 먹여 요약하는 시스템이 아니라, 결정론적 정적 분석 결과를 코드 구조와 위협 지식 맥락으로 재구성한 뒤, 에이전트가 근거 중심으로 취약점 후보를 판단하고 향후에는 PoC 및 동적 검증까지 수행하도록 설계된 embedded security analysis platform이다.

연구 소개용 표현

AEGIS의 연구적 핵심은 "더 많은 취약점을 찾는 것"보다 "근거를 추적 가능한 취약점 주장을 만들고, 이를 exploitability 수준까지 검증 가능한 구조로 확장하는 것"에 있다.

---

13. 최종 요약

AEGIS는 다음 한 문장으로 요약할 수 있다.

> **AEGIS는 임베디드 코드 분석을 위해 결정론적 정적 분석, 보안 지식 그래프, 사례 검색, 에이전트 reasoning, 그리고 장기적으로 동적 확인까지 연결하는 근거 중심 분석 플랫폼이다.**

그리고 논문화는 다음처럼 가는 것이 가장 자연스럽다.

- **논문 A1:** 근거 강제형 deterministic-first embedded vulnerability triage
- **논문 A2:** 정적 후보를 동적 확인으로 연결하는 embedded exploitability confirmation

이렇게 잡으면 AEGIS는 단순히 "기능이 많은 플랫폼"이 아니라,
서로 다른 두 개의 명확한 연구 주장으로 분해 가능한 연구 프로그램이 된다.